

Universidad de la Sierra Sur

Monitoreo y resiliencia de la Gaceta Universitaria «El Unsito» con Amazon S3, AWS Lambda y CloudWatch

F I C H A T É C N I C A

PROYECTO INTEGRADOR · EQUIPO 3
TECNOLOGÍAS DE LA INFORMACIÓN I

Presentan:

Daniel González Ruiz
Mayra Castellanos Pacheco
Gael Eduardo García Pinacho
Elisa Sánchez Martínez

Docente:

M.C.C. Eliezer Alcazar Silva

Miahuatlán de Porfirio Díaz, Oaxaca.
Octubre de 2026

Índice general

1. Introducción **1**

1.1. Problema 1

1.2. Objetivo 1

1.3. Alcance 1

2. Arquitectura **2**

2.1. Antes y ahora 2

2.2. Recorrido de una publicación 3

2.3. Recursos 4

3. Monitoreo y resiliencia **5**

3.1. Señal observable 5

3.2. Manejo de fallos 5

3.3. Resultados de las pruebas 6

4. Seguridad y costos **8**

4.1. Mínimo privilegio 8

4.2. Costos 8

4.3. Recursos por eliminar 9

5. Conclusiones **10**

Índice de figuras

2.1 Arquitectura antes y después de la integración 2

2.2 Secuencia de una publicación con imagen 3

3.1 Decisiones de la procesadora ante cada evento y prueba que valida cada camino. 6

Capítulo 1

Introducción

1.1. Problema

La gaceta ya permitía publicar noticias con imagen, pero el navegador coordinaba todo el proceso: primero subía la imagen a Amazon S3 y después llamaba a la función `agregarNoticia` para guardar la noticia en DynamoDB. Si la segunda llamada fallaba, la imagen quedaba almacenada sin noticia y no existía ningún registro, métrica ni aviso del error.

1.2. Objetivo

Que la nube procese la publicación por sí misma en cuanto llega la imagen, y que cualquier fallo quede registrado, se reintente cuando tenga sentido y genere una alerta. En concreto, el equipo se propuso:

- Conectar el evento de carga de S3 con una función Lambda y retirar la llamada que hacía el navegador después de subir la imagen.
- Registrar logs útiles de esa función sin incluir secretos.
- Definir una señal observable —la métrica de errores— con una alarma.
- Documentar qué ocurre ante una carga fallida, una imagen sin datos y un error al guardar.

1.3. Alcance

El trabajo se hizo en la región us-east-2 (Ohio) sobre el bucket y la tabla que comparte todo el grupo, sin modificar su estructura. Las noticias sin imagen siguen usando la función `agregarNoticia`, porque no generan un evento de S3. La cola de mensajes fallidos con SQS, propuesta como ampliación opcional, no se implementó.

Capítulo 2

Arquitectura

2.1. Antes y ahora

El cambio principal es quién inicia el guardado. Antes lo hacía el navegador con una segunda llamada; ahora lo hace S3, que avisa a una función nueva en cuanto termina de recibir la imagen (figura 2.1).

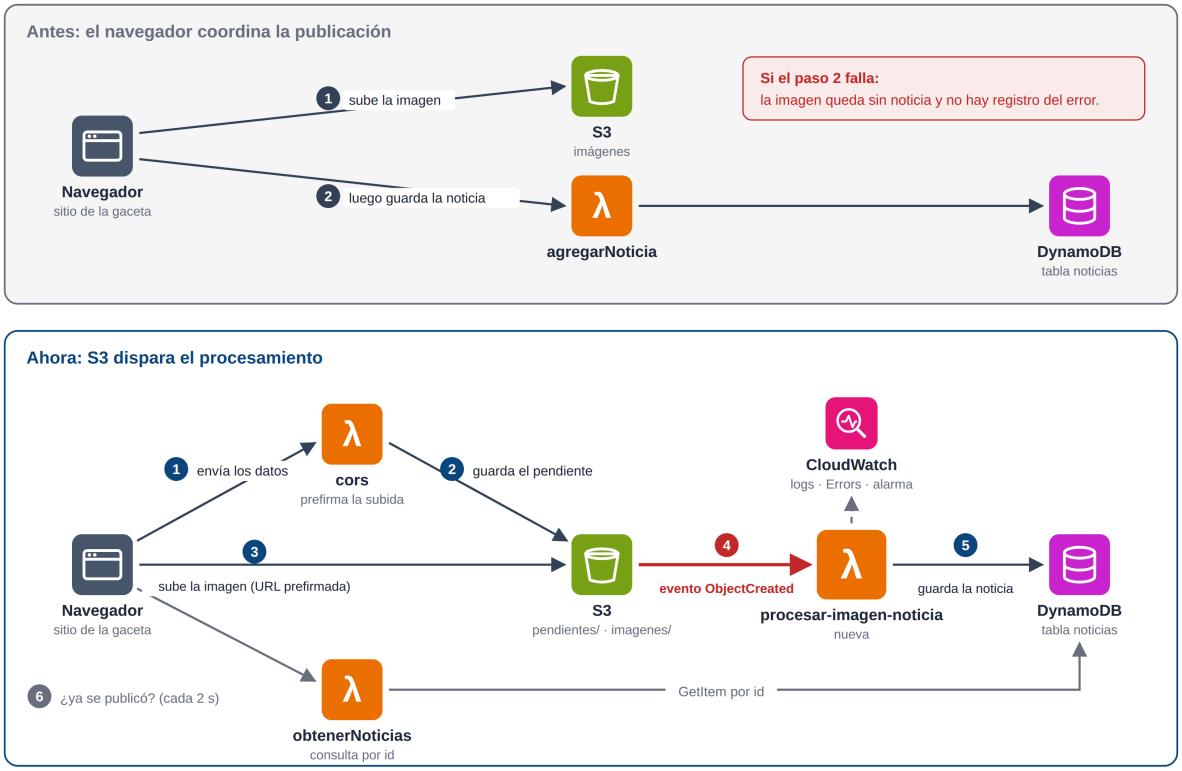


Figura 2.1: Arquitectura antes y después de la integración. La flecha roja es el evento de S3 que ahora inicia el procesamiento.

2.2. Recorrido de una publicación

La web envía primero los datos de la noticia. La función `cors` los guarda como «pendiente» y devuelve un permiso temporal para subir la imagen. Al llegar la imagen, S3 invoca a `procesar-imagen-noticia`, que lee el pendiente, lo valida y guarda la noticia. Mientras tanto, la web pregunta cada dos segundos si la noticia ya está publicada (figura 2.2).

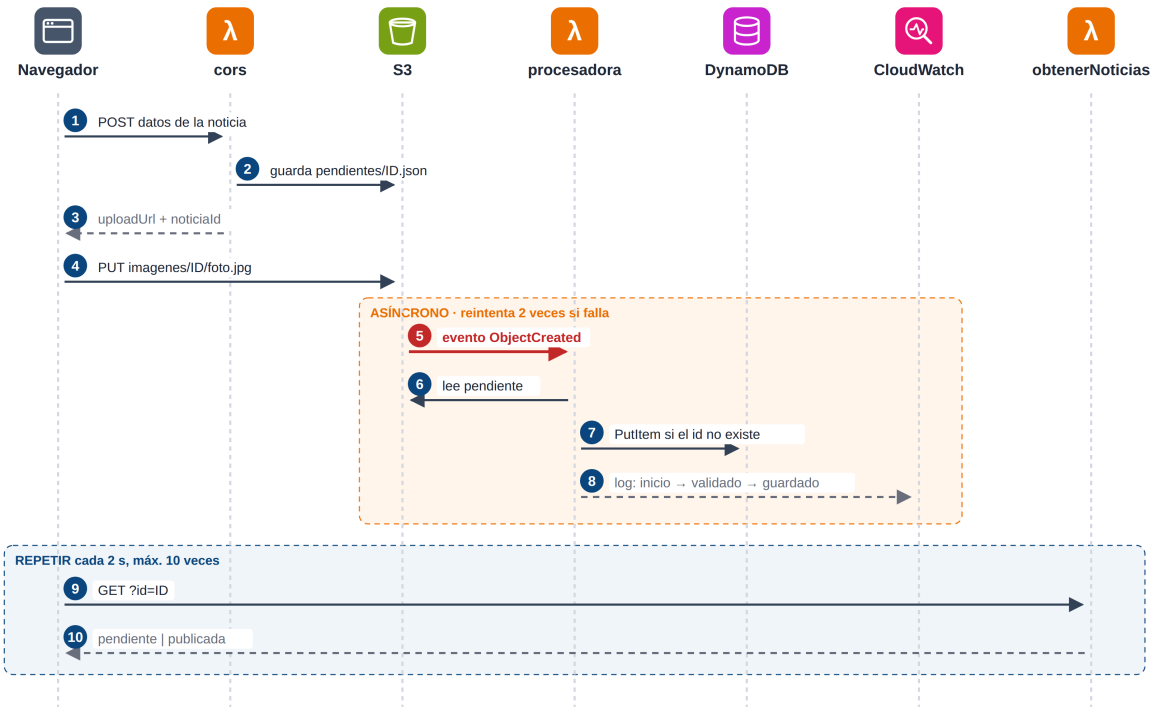


Figura 2.2: Secuencia de una publicación con imagen. El tramo naranja es asíncrono: si falla, Lambda lo reintenta.

2.3. Recursos

Tabla 2.1: Recursos que intervienen en el flujo.

Recurso	**Nombre**	**Función**
Bucket S3	`mis-noticias-daniel-imagenes-906-2026`	Privado. Guarda las imágenes y los pendientes
Tabla DynamoDB	`noticias`	Noticias publicadas (clave `id`)
Lambda	`cors`	Guarda el pendiente y firma la subida
Lambda (nueva)	`procesar-imagen-noticia`	Procesa el evento de S3 y guarda la noticia
Lambda	`obtenerNoticias`	Lista noticias y consulta el estado por id
CloudWatch	`gaceta-procesadora-errores`	Alarma sobre los errores de la procesadora

Capítulo 3

Monitoreo y resiliencia

3.1. Señal observable

Se eligió la métrica `Errors`, que Lambda publica en CloudWatch sin configuración adicional. La alarma `gaceta-procesadora-errores` pasa a estado de alarma cuando hay al menos un error en un minuto. Además, la procesadora escribe una línea de log por etapa (`inicio`, `validado`, `guardado`, `duplicado`, `fallo`) con el identificador de la noticia, lo que permite rastrear cualquier publicación. Los logs no incluyen firmas de URL ni el texto de la noticia, y se conservan tres días.

3.2. Manejo de fallos

No todos los errores se tratan igual. Los que no se arreglan con reintentos terminan sin error y se registran; los que pueden ser temporales, como un fallo de la base de datos, se relanzan para que Lambda los reintente y la alarma se active (figura 3.1).

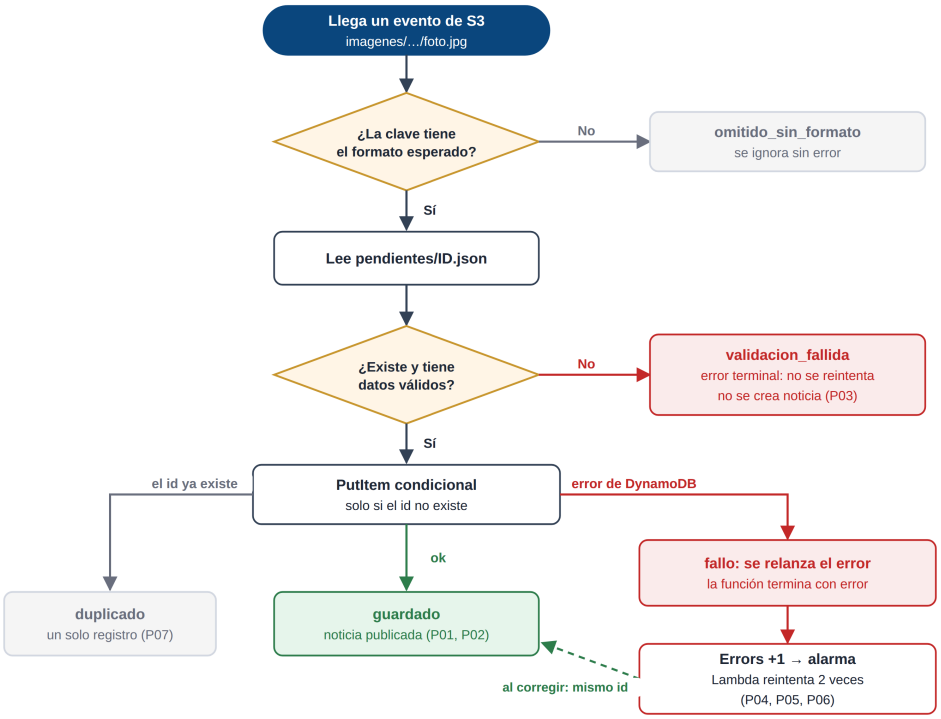


Figura 3.1: Decisiones de la procesadora ante cada evento y prueba que valida cada camino.

Tabla 3.1: Qué hacer ante los fallos que pide la rúbrica.

Situación	Qué hace el sistema	Qué hace el equipo
Carga fallida	Sin imagen no hay evento, así que no se guarda nada a medias. La web conserva los datos y reintenta con el mismo id.	Volver a publicar desde el formulario.
Imagen sin datos	Registra `validacion_fallida` y termina; no se crea la noticia.	Buscar el id en los logs y publicar de nuevo.
Error al guardar en DynamoDB	Registra `fallo`, reintenta dos veces y enciende la alarma.	Corregir la causa y reprocesar el evento; no se duplica.

3.3. Resultados de las pruebas

Se ejecutaron ocho pruebas el 27 de octubre de 2026 y todas se aprobaron. Las capturas de cada una están en el manual de implementación.

Tabla 3.2: Pruebas de funcionamiento y de error controlado.

Prueba	**Resultado**	**Estado**
P01 Imagen válida	El evento de S3 activó la procesadora, sin llamada a `agregarNoticia`	**Aprobada**
P02 Persistencia	La noticia quedó en DynamoDB y se ve al recargar el sitio	**Aprobada**
P03 Imagen sin datos	Se registró `validacion_fallida` y no se creó la noticia	**Aprobada**
P04 Falla DynamoDB	Tres intentos fallidos; al corregir se guardó con el mismo id	**Aprobada**
P05 Métrica Errors	La métrica registró el pico de errores	**Aprobada**
P06 Alarma	Se encendió a las 23:46 y volvió a la normalidad a las 23:51	**Aprobada**
P07 Evento duplicado	Se registró `duplicado`; quedó un solo registro	**Aprobada**
P08 Proceso lento	La web mostró «en procesamiento» sin reenviar la noticia	**Aprobada**

Capítulo 4

Seguridad y costos

4.1. Mínimo privilegio

La procesadora tiene su propio rol, `procesar-imagen-noticia-rol`, con una sola política que permite únicamente lo que la función usa. Solo S3 puede invocarla, y únicamente desde el bucket de imágenes.

Tabla 4.1: Permisos del rol de la procesadora.

Puede	**Sobre**	**No puede**
Leer los datos pendientes	S3, carpeta `pendientes/`	Leer las imágenes ni el resto del bucket
Guardar noticias	Tabla `noticias`	Leer, borrar ni recorrer la tabla
Escribir logs	Su propio grupo de logs	Escribir en otros grupos

El código y las variables de entorno no contienen claves. Las otras tres funciones del proyecto siguen con un rol amplio (`PowerUserAccess`); en un entorno real cada una debería tener el suyo.

4.2. Costos

El saldo fue de 120.00 USD antes y después de las pruebas, con un gasto de 0.00 USD: Lambda, S3, DynamoDB bajo demanda y CloudWatch quedaron dentro del nivel gratuito. Para no generar cargos se limitó la retención de logs a tres días, la alarma no envía correos ni SMS y no se crearon colas ni servidores. Los recursos del equipo llevan las etiquetas `proyecto=NoticiasAWS`, `equipo=3` y `entorno=Laboratorio`.

4.3. Recursos por eliminar

Al cerrar el curso se eliminan, en este orden: el disparador de S3, la función `procesar-imagen-noticia`, su rol, la alarma, el grupo de logs, los objetos de `pendientes/` y las imágenes de prueba, y las noticias de prueba de la tabla. Si el sitio sigue en uso, antes se restaura la versión original de la función `cors`.

Capítulo 5

Conclusiones

La publicación de noticias ya no depende de que el navegador complete una segunda llamada. S3 inicia el procesamiento, cada noticia deja un rastro en los logs, los fallos temporales se reintentan solos y una alarma avisa cuando algo sale mal. Las ocho pruebas confirman el funcionamiento normal y el comportamiento ante errores, sin duplicar noticias.

Para un entorno real quedarían pendientes una cola de mensajes fallidos con SQS, el envío de la alarma por correo con SNS, la eliminación automática de los pendientes antiguos y un rol de mínimo privilegio para cada función del proyecto.